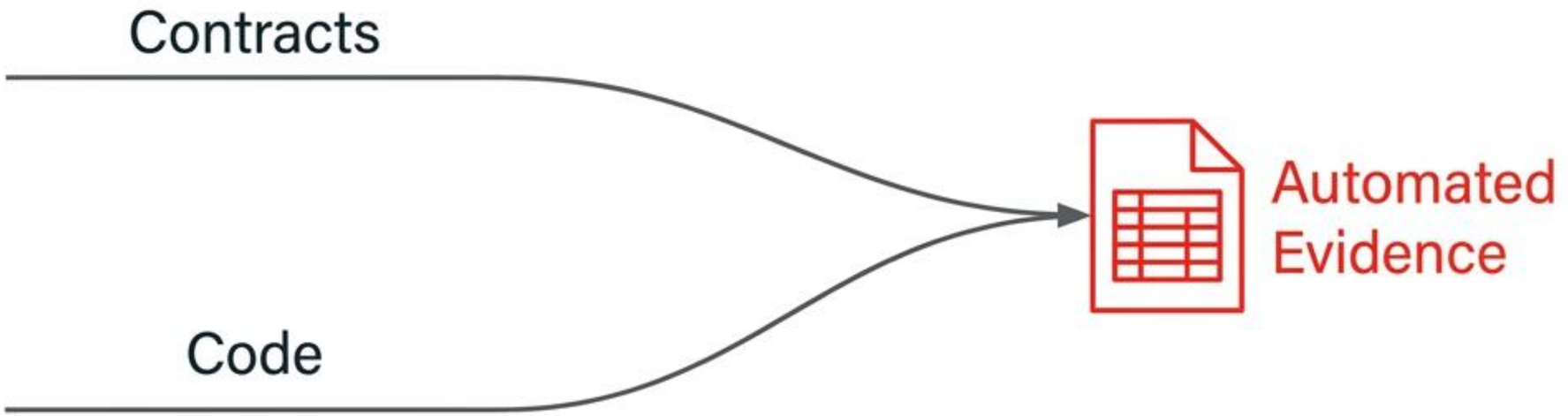
The Governance & Lineage Engine: Evidence that generates itself

Data contracts + column-level lineage → an automated compliance report that catches undeclared PII.



Governance is done by hand, and by hand it fails.



Code Merged

Lineage drawn in slides

Stale the day after publication.

PII discovered in the wrong place.

Found during an audit, not before.

Where does this data go, and is it classified correctly? takes humans days to answer. Manual governance is an archaeology project.

Harvest governance from the pipeline; don't document it by hand.

Contracts

Declare what should be true (schema, classification, AI-usage).

+

Lineage

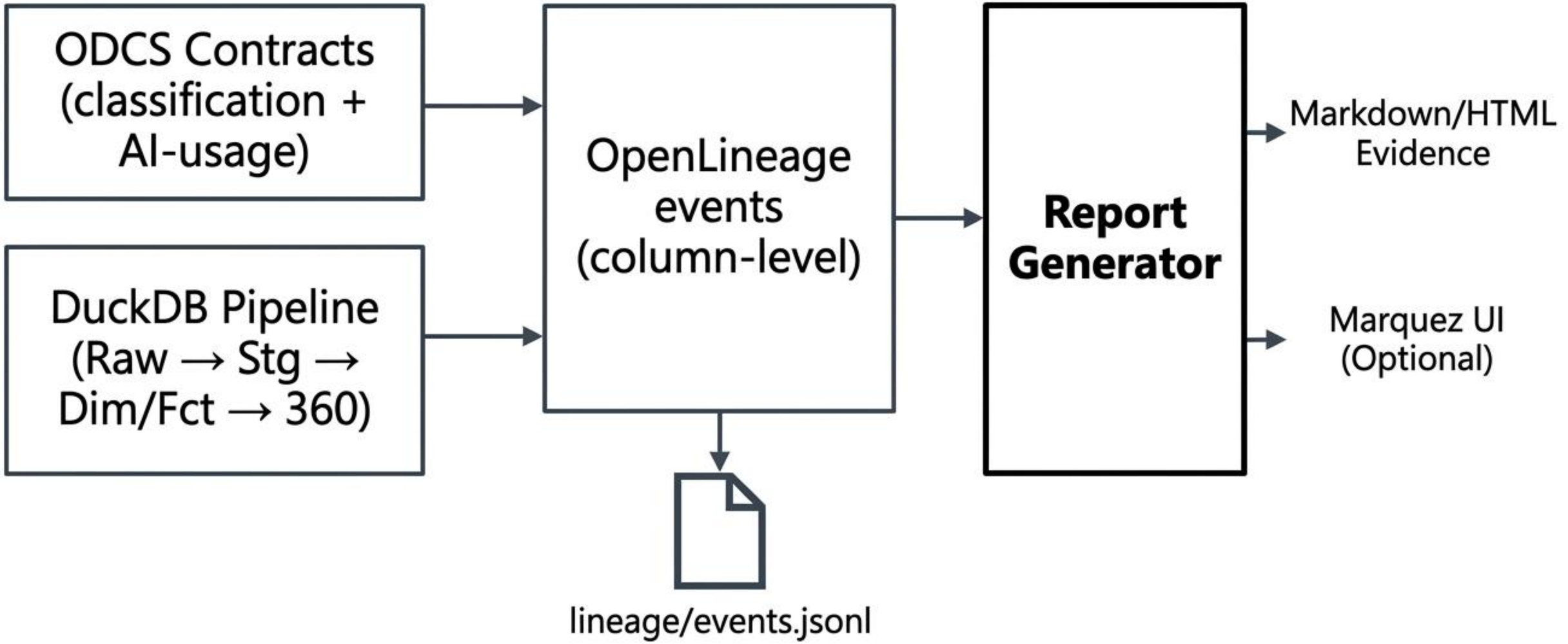
Record what actually happened, column by column.

=

The Generator

Reconciles the two → a compliance report, automatically.

Architecture: Three inputs, one evidence pack.



Four ODCS contracts, with classification and AI-usage.

```
table: dim_customer
columns:
  - name: email
  - name: dob
  - name: full_name
  - name: tax_file_number   [PII / Critical / Restricted]


table: customer_360   [v3.2.0 AI-Usage]
```

1. PII is tagged at the column level.

2. AI-usage fields declare how customer_360 may (and may not) be used by downstream models.

The deliberate gap: customer_360 carries PII it never declared.



A realistic, dangerous, everyday governance failure—planted on purpose.
A realistic, dangerous, everyday governance failure—planted on purpose.
The mart aggregates a 360 view, but inherits sensitive fields nobody re-classified.

The generated report flags the undeclared PII — by itself.

Governance Gaps

Field	Status	Lineage Source
customer_360.email	HIGH GAP	dim_customer
customer_360.full_name	HIGH GAP	dim_customer

No human wrote “there’s a problem here.” The engine reconciled the declared classification against the actual lineage and found the gap via make demo.

The clever bit: Classification follows the data, but only where it should.

Direct / Identity Transform

Data: email

↓

Action: Carried over exactly as-is.

Outcome: PII Propagated downstream. Flags raised.

Aggregation Transform

Data: amount

↓

Action: Summed into total_spend.

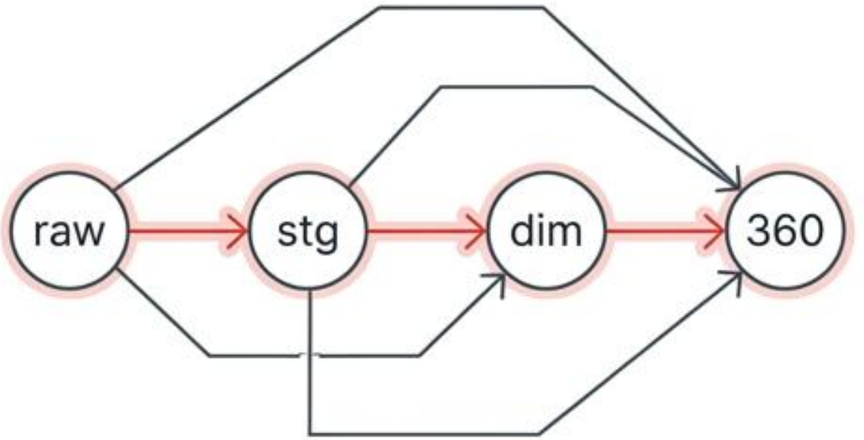
Outcome: PII Dropped. Safe. No false alarms.

Note: tax_file_number is correctly dropped before the mart → correctly not propagated. Accuracy over a blunt rule.

Lineage you can see:
OpenLineage events → an interactive graph.

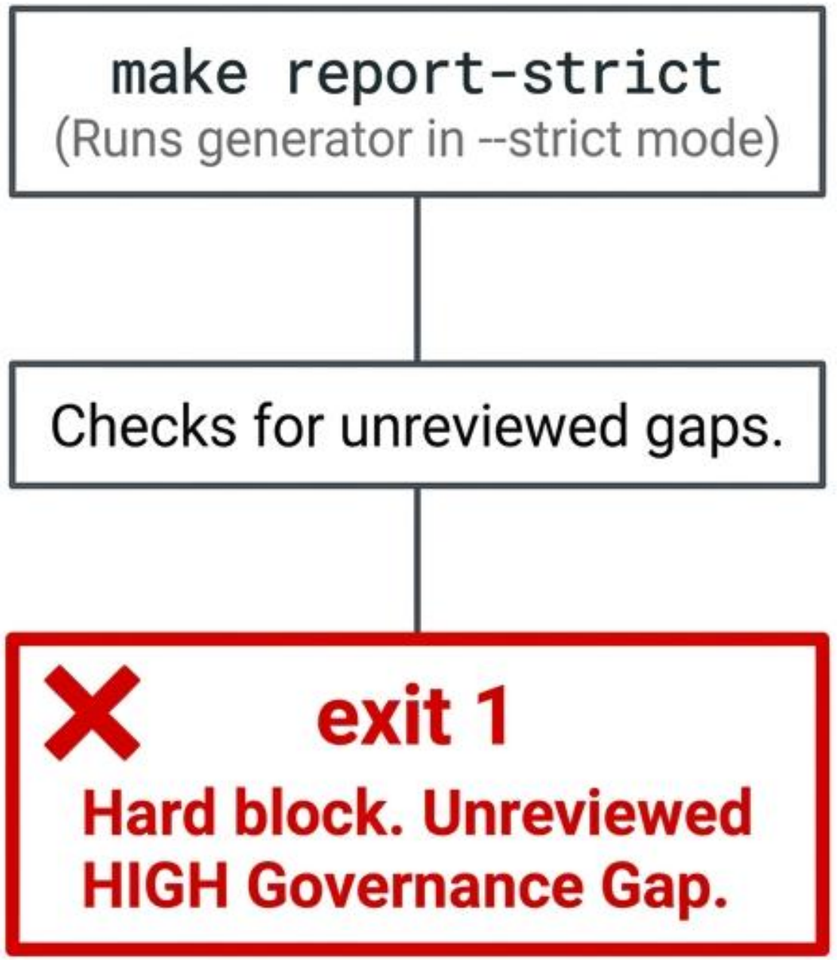
```
{
  "eventType": "COMPLETE",
  "run": { "runId": ".." },
  "facets": {
    "columnLineage": {
      "fields": {
        "email": {"inputFields": […]}
      }
    }
  }
}
```

make marquez-up →



One source of truth. The exact same events that power the compliance report also power the visual lineage UI.

The CI gate: Governance that blocks a merge.



Governance stops being a quarterly review and becomes a **per-change control**.

Engineering decisions & trade-offs: Choices a reviewer should trust.

Hand-wrote OpenLineage events

Why: Transparency and zero version coupling, rather than leaning blindly on a client lib.

Smart Propagation

Why: Distinguishes carry vs aggregite for accuracy, avoiding blunt rules.

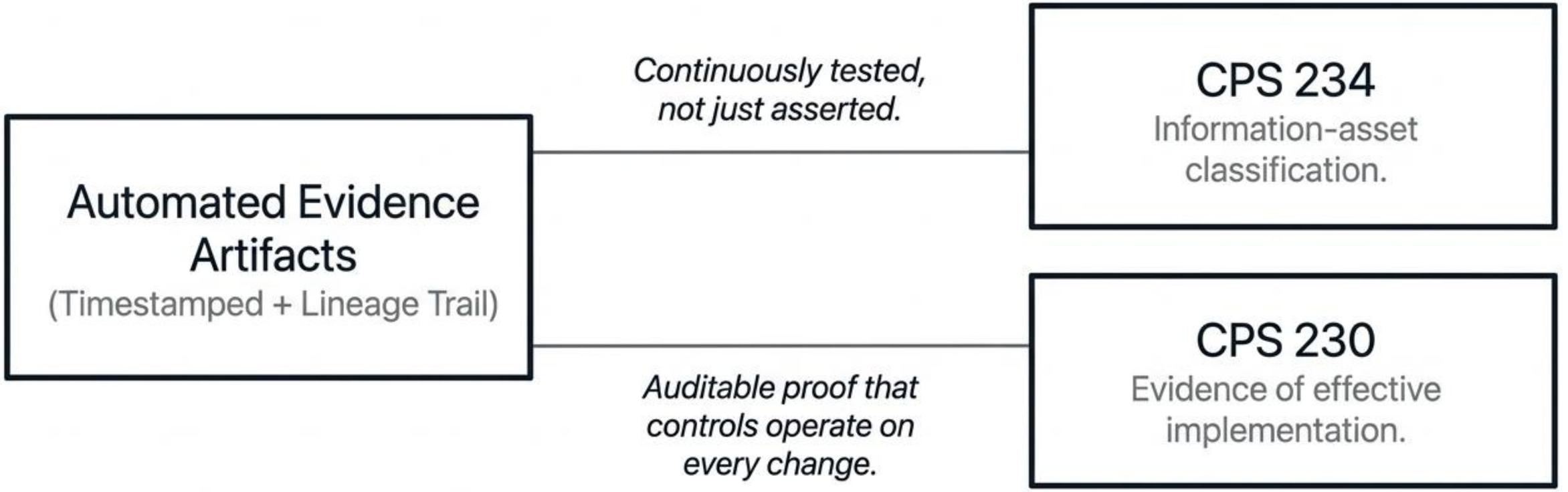
Two Runtimes

Why: Standalone Python script and Airflow DAG, proving identical core logic across orchestrators.

DuckDB + File-based lineage

Why: Fully reproducible locally in minutes, no cloud infrastructure required.

Regulatory mapping: Automated evidence for modern prudential expectations.



General information, not compliance advice.

What's simplified, and why.

Simplified for Demo	Production-Real
Column lineage is declared in transforms, not parsed dynamically from arbitrary SQL.	The reconciliation logic.
Small synthetic dataset.	The mechanism comparing contracts vs actual lineage to detect gaps.
Marquez UI is optional.	The CI/CD enforcement mechanism.

Takeaway: The governance reasoning on top of the lineage is the core contribution, and that is real.

Clone, run, and verify it locally.

```
make setup

make demo # pipeline → lineage → compliance report

make report-strict # ❌ exit 1 — CI gate on unreviewed PII
```

OpenLineage | Column-level lineage | ODCS classification

Compliance automation | Airflow | Python | DuckDB | CPS 230/234

The capstone of a 3-project shift-left story.



Repo: github.com/initiative1972/shift-left-3-governance-lineage |

Full Portfolio: github.com/initiative1972 | Henry Yan · AI Engineer & Data Architect · Melbourne